



Sri Lanka Institute of Information Technology

Data Warehousing and Business Intelligence IT3021

Hotel Booking Analysis Data Warehouse Submission Report

Assignment 1

2026

Student ID : IT23194908

Student Name : R A S R Harischandra

Batch / Group : Y3.S2.DS.WE.02.02

Table of Contents

1	Dataset Selection – Hotel Booking Analytics	3
1.1	Why this is OLTP ?	3
1.2	Why Dataset is Suitable ?	3
2	Preparation of data sources	5
2.1	Creating Data Sources for HotelDW	5
2.2	Data Source Types Used	5
3	Solution Architecture	6
4	Data Warehouse Design and Development	8
4.1	Schema Used – Star Schema	8
4.2	Dimension and Fact Tables	8
4.3	Assumptions.....	8
4.4	SQL Implementation	10
5	ETL Development	12
5.1	ETL Overview	12
5.2	SSIS Project Structure.....	12
5.3	Load Staging Tables.....	13
5.4	Load Dimensions Tables	17
5.5	Load Fact Table.....	21
6	Accumulating Fact Table.....	22
6.1	Fact Table Analysis	22
6.2	Accumulating Columns & Update	22
6.3	Creation of Completion Dataset	23
6.4	SSIS Package for Updating Fact Table	24
6.5	Updating Completion Time in Fact Table.....	25
6.6	Calculating Processing Time	25
6.7	Validation of Accumulating Fact Data	26

1 Dataset Selection – Hotel Booking Analytics

Dataset selected – [Hotel Booking Analytics](#)

This dataset represents raw hotel booking transactions collected from a real-world reservation environment. It includes information for both City Hotels and Resort Hotels, covering operational activities such as:

- Customer details (user ID, gender, country, age group, traveler type, join_date)
- Booking information (hotel ID, hotel name, city, country, star rating, stay dates)
- Review and rating details (Review ID, overall score, cleanliness, comfort, facilities, location, staff, value for money)
- Transactional timestamps (booking ID, booking date, review date)

The dataset is provided in its original OLTP form, meaning the data is stored in a transactional structure rather than pre-aggregated fact and dimension tables. This makes it ideal for ETL tasks, dimensional modeling, and building a Data Warehouse.

1.1 Why this is OLTP ?

The dataset reflects an Online Transaction Processing (OLTP) system as it captures day-to-day operations of a hotel review platform. Each review represents a transaction involving a user and a hotel at a specific point in time. The data is highly normalized and focuses on capturing detailed records rather than aggregated information, making it suitable for transformation into a data warehouse.

1.2 Why Dataset is Suitable ?

The dataset is suitable for data warehouse implementation due to the following reasons:

- It consists of multiple data sources (CSV and Excel formats)
- It includes time-based data (review_date) enabling trend analysis
- It contains multiple entities allowing dimensional modeling
- It provides sufficient attributes to create dimensions and measures
- It can be extended to include large volumes of data for analysis

Total Reviews	~50,000
Source Types	CSV, Excel
Entities	Hotels, Users, Reviews
Measures	Ratings(cleanliness, comfort)
Time Attribute	Review date
Locations	Multiple cities & countries

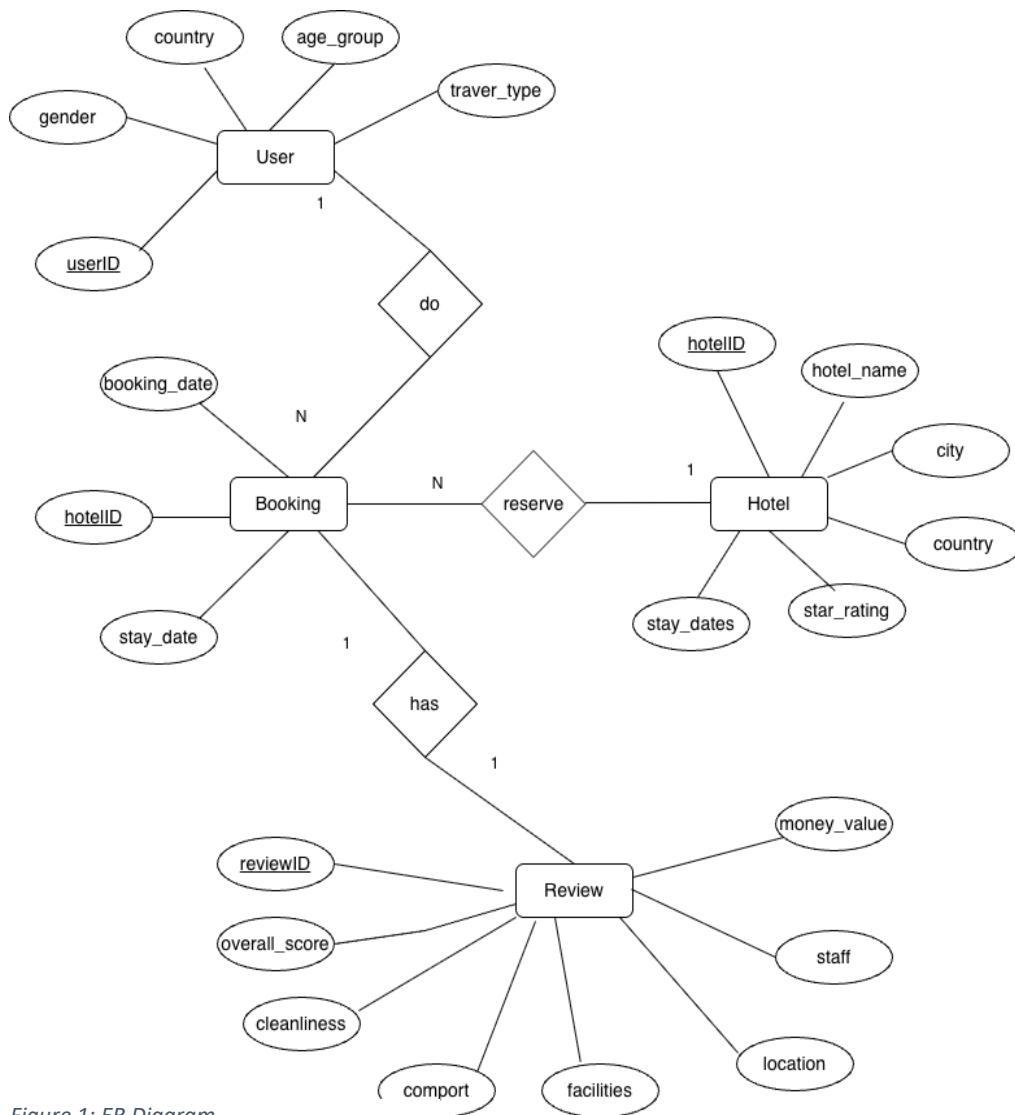


Figure 1: ER Diagram

2 Preparation of data sources

2.1 Creating Data Sources for HotelDW

To prepare the dataset for ETL processing, the original hotel booking data was **separated into multiple source types** and imported into the SQL Server environment. This ensures that the system simulates a real-world scenario where data originates from different formats.

2.2 Data Source Types Used

1. CSV Files (Flat Files)

The following datasets were stored as **CSV files** and imported using the **BULK INSERT method / Flat File Source in SSIS**:

- **Hotels (hotel_v1.csv)**
 - hotel_id, hotel_name, city, country, star_rating, lat, lon
 - cleanliness_base, comfort_base, facilities_base, location_base, staff_base, value_for_money_base
- **Reviews (reviews_v1.csv)**
 - review_id, user_id, hotel_id, review_date
 - score_overall, score_cleanliness, score_comfort, score_facilities, score_location, score_staff, score_value_for_money, review_text

2. Excel File

The user-related dataset was stored as an **Excel file** and imported using the **Import Data Wizard / Excel Source in SSIS**:

- **Users (users_v1.xls)**
 - user_id, user_gender, country, age_group, traveller_type, join_date

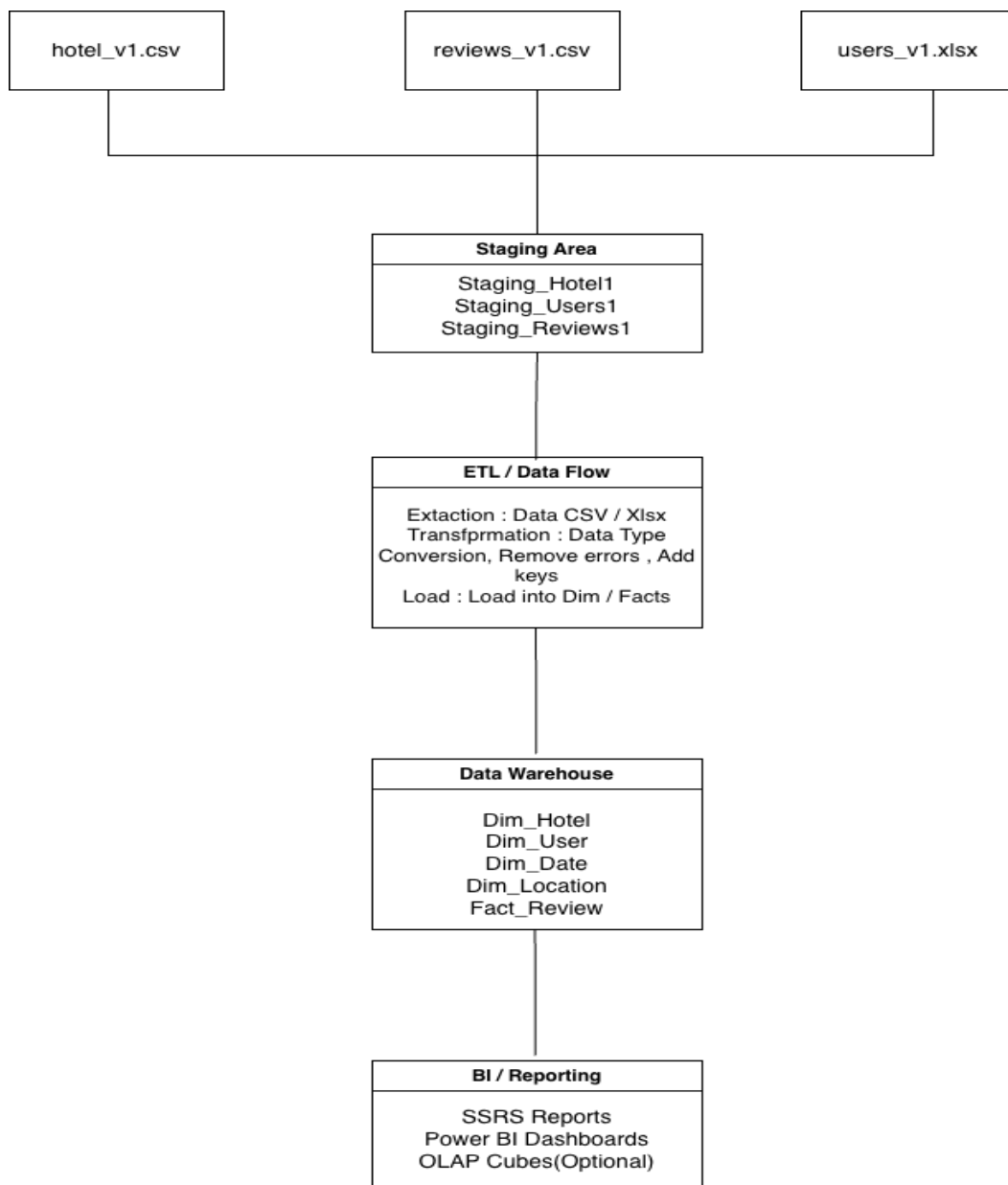
3. Database (SQL Server – Staging Layer)

All extracted data from CSV and Excel sources were loaded into **staging tables** in the SQL Server database (HotelDW):

- Staging_Hotels
- Staging_Reviews
- Staging_Users

Data Source	Format	Content	Target Usage
hotel_v1.csv	CSV	Hotel details & base ratings	Dim_Hotel, Dim_Location
reviews_v1.csv	CSV	Review transactions & scores	Fact_Review, Dim_Date
users_v1.xls	Excel	User demographic data	Dim_User
SQL Server (Staging)	Database	Cleaned & structured data	Final DW tables

3 Solution Architecture



The following describes the high-level architecture designed to process hotel review data into actionable business insights. The architecture is composed of five distinct layers:

1. Data Sources

The foundation of the project involves gathering raw data from multiple formats to ensure a comprehensive view of the business. In this scenario, three primary data sources are utilized:

- `hotel_v1.csv`: A flat file containing descriptive information about hotel entities, including names, star ratings, and geographical coordinates.
- `reviews_v1.csv`: A flat file containing the transactional review data, including various scores (cleanliness, comfort, value) and timestamps.
- `users_v1.xlsx`: An Excel spreadsheet containing demographic details of the customers, such as age groups and traveler types.

2. Staging Area

Before data is moved into the permanent warehouse, it is loaded into a Staging Area (e.g., `Staging_Hotel1`, `Staging_Users1`).

- This acts as a temporary landing zone to minimize the performance impact on source systems.
- It allows for a consolidated environment where data from different formats (.csv and .xlsx) can be aligned before the transformation process begins.

3. ETL / Data Flow (The Processing Layer)

The ETL (Extract, Transform, Load) process is the engine of the architecture. In this solution, tools like SQL Server Integration Services (SSIS) are typically used to perform the following:

- **Extraction:** Pulling data from the staging tables.
- **Transformation:** This is the most critical step, involving data type conversion, removing null or erroneous values (data cleansing), and generating surrogate keys for the dimensions.
- **Load:** Moving the refined data into the final Star Schema structure of the Data Warehouse.

4. Data Warehouse (HotelDW)

The Data Warehouse layer represents the "Single Version of Truth" for the organization. It is organized into a Star Schema to optimize query performance:

- **Dimension Tables:** `Dim_Hotel`, `Dim_User`, `Dim_Date`, and `Dim_Location` store the descriptive attributes (the "who, where, and when").
- **Fact Table:** `Fact_Review` stores the quantitative measures and foreign keys, allowing for complex multi-dimensional analysis.

5. BI / Reporting (Consumption Layer)

This is the final layer where data is converted into visual intelligence for stakeholders.

- Power BI Dashboards: Used for real-time, interactive visualization of hotel performance and customer satisfaction trends.
- SSRS Reports: Utilized for structured, paginated operational reporting.
- OLAP Cubes (Optional): Provided via SSAS to allow for high-speed "slice and dice" analysis across large volumes of historical data.

4 Data Warehouse Design and Development

4.1 Schema Used – Star Schema

A Star Schema is used in this design to simplify querying and improve performance for analytical processing.

All dimension tables are directly connected to the central fact table, forming a star-like structure. Unlike snowflake schema, no normalization is applied to dimension tables, ensuring faster query execution and easier understanding.

The central fact table, Fact_Review, stores hotel review transactions and includes multiple rating measures. It is connected to all dimension tables using foreign keys.

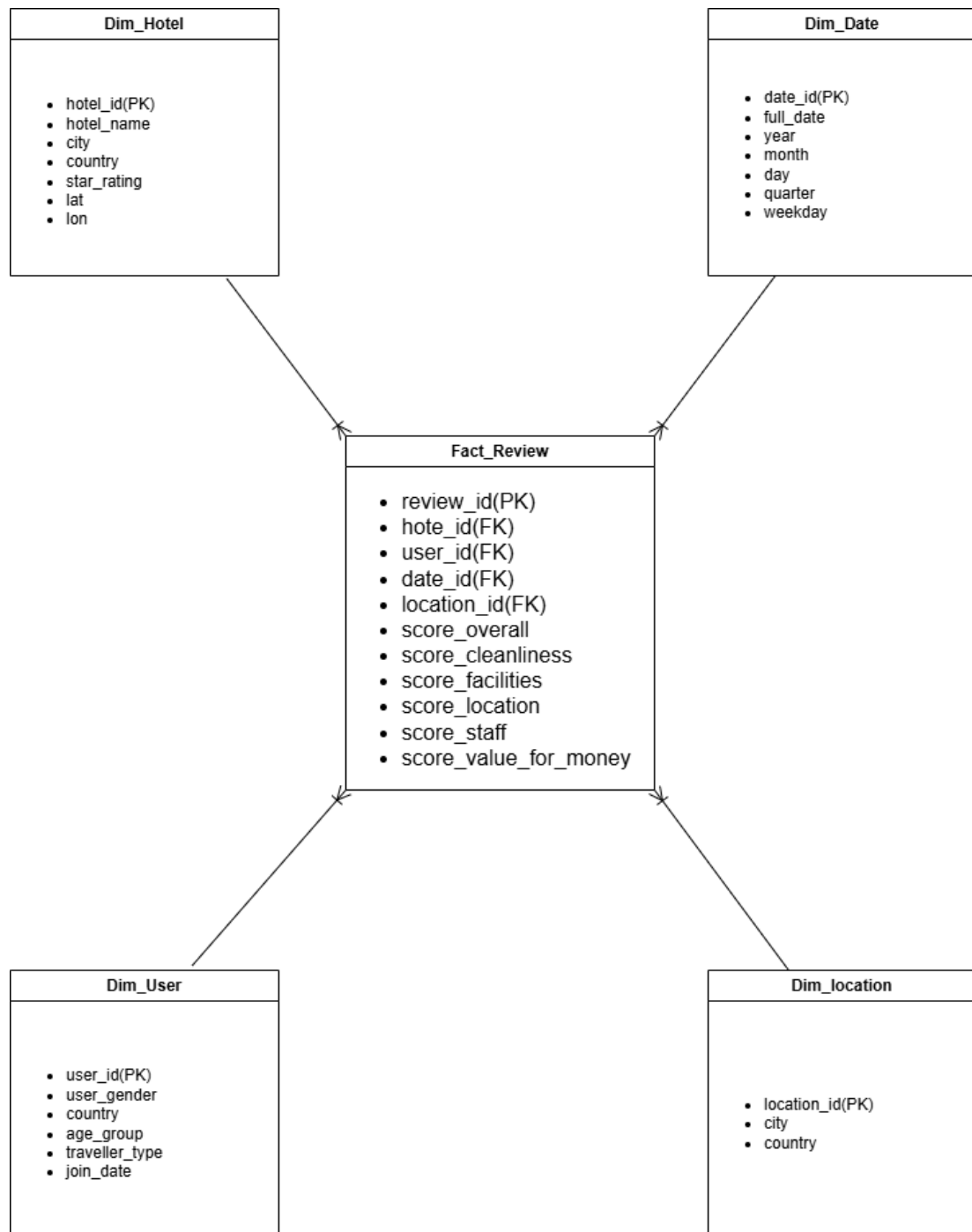
4.2 Dimension and Fact Tables

Four dimension tables (including Date) and one fact table were created:

- Dim_Hotel – Stores hotel details (hotel_id, hotel_name, city, country, star_rating, lat, lon).
Primary Key: hotel_id
- Dim_User – Stores user details (user_id, gender, country, age_group, traveller_type, join_date).
Primary Key: user_id
- Dim_Location – Stores location data (location_id, city, country).
Primary Key: location_id
Hierarchy: Country → City
- Dim_Date – Time dimension (date_id, full_date, year, month, day, quarter, weekday).
Primary Key: date_id
- Fact_Review – Stores review transactions.
Primary Key: review_id
Foreign Keys: hotel_id, user_id, date_id
Measures: overall, cleanliness, comfort, facilities, location, staff, value_for_money

4.3 Assumptions

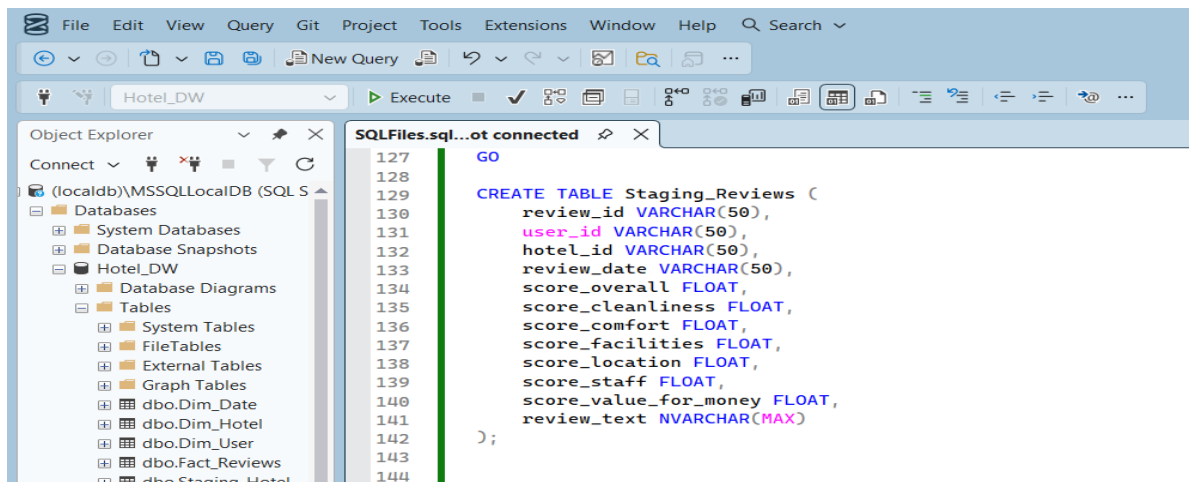
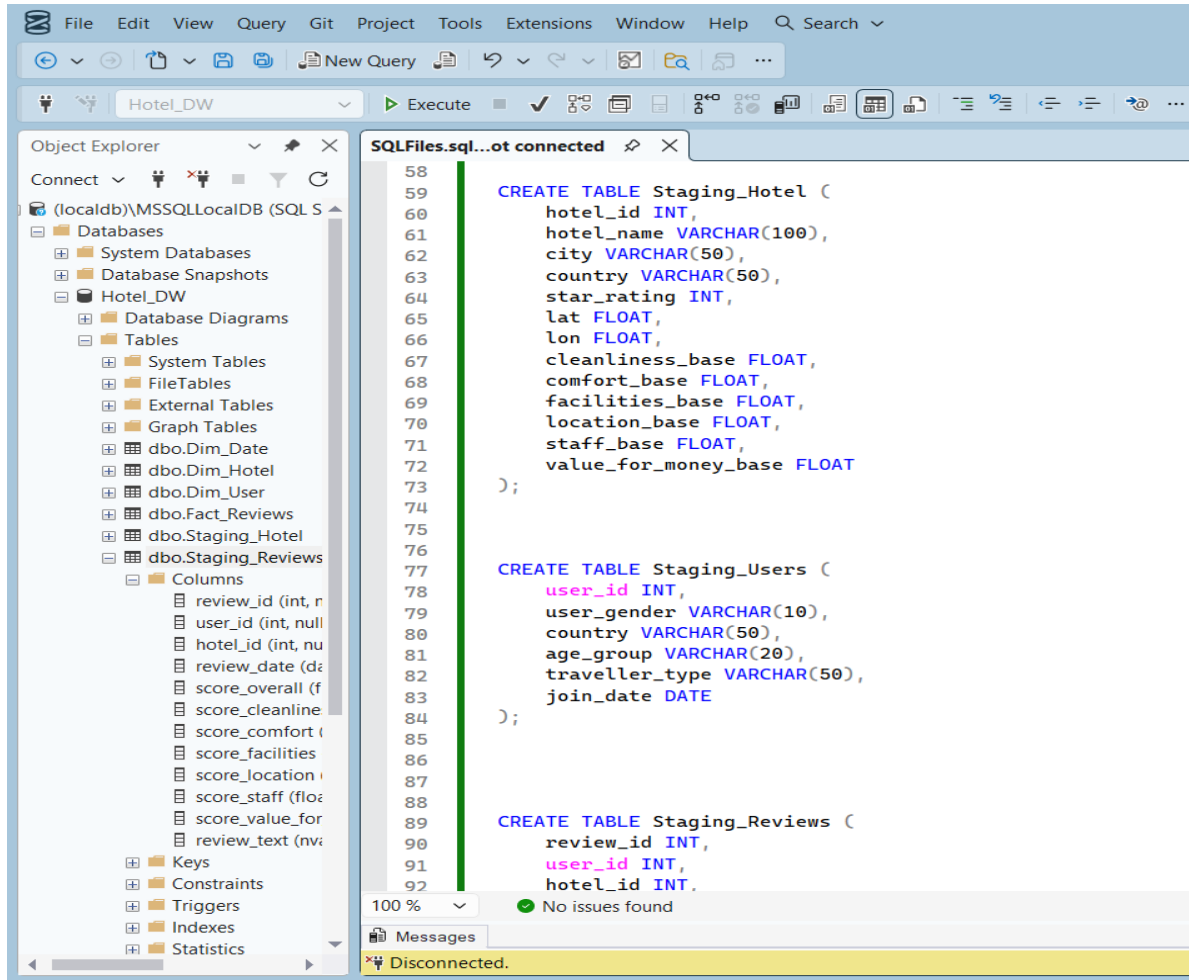
- One review belongs to one user and one hotel
- Each review has a single date
- Scores are numeric and aggregatable
- Location is separated for hierarchy
- Data is cleaned during ETL

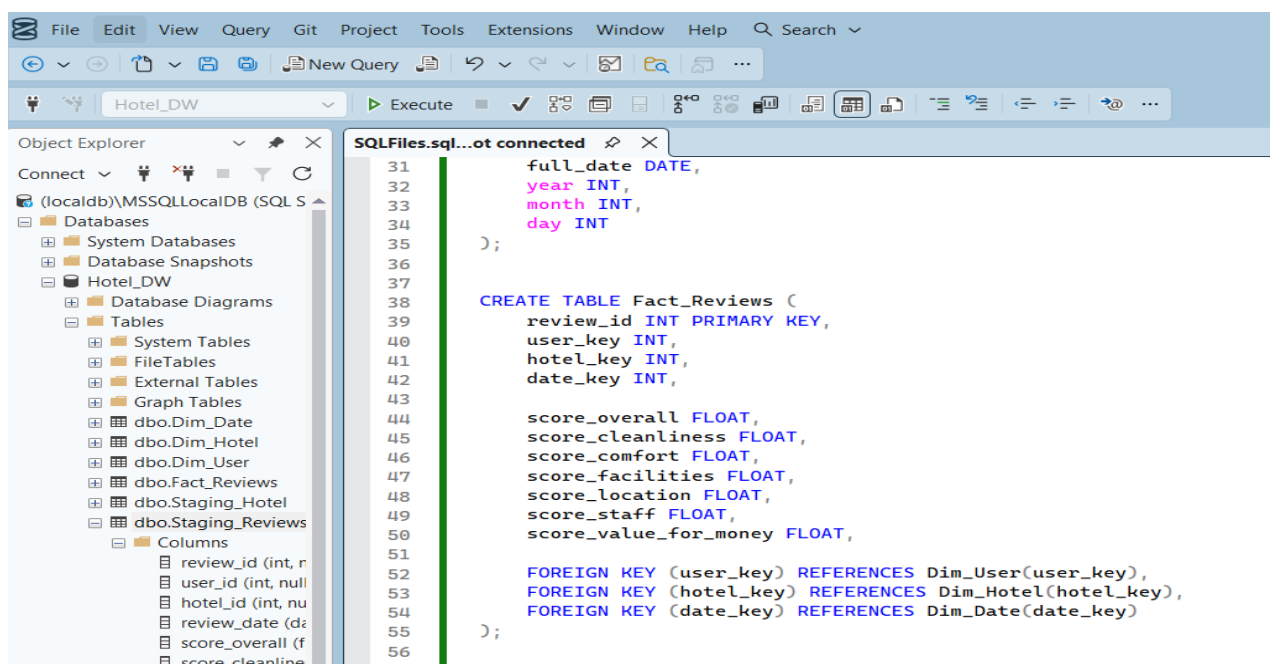
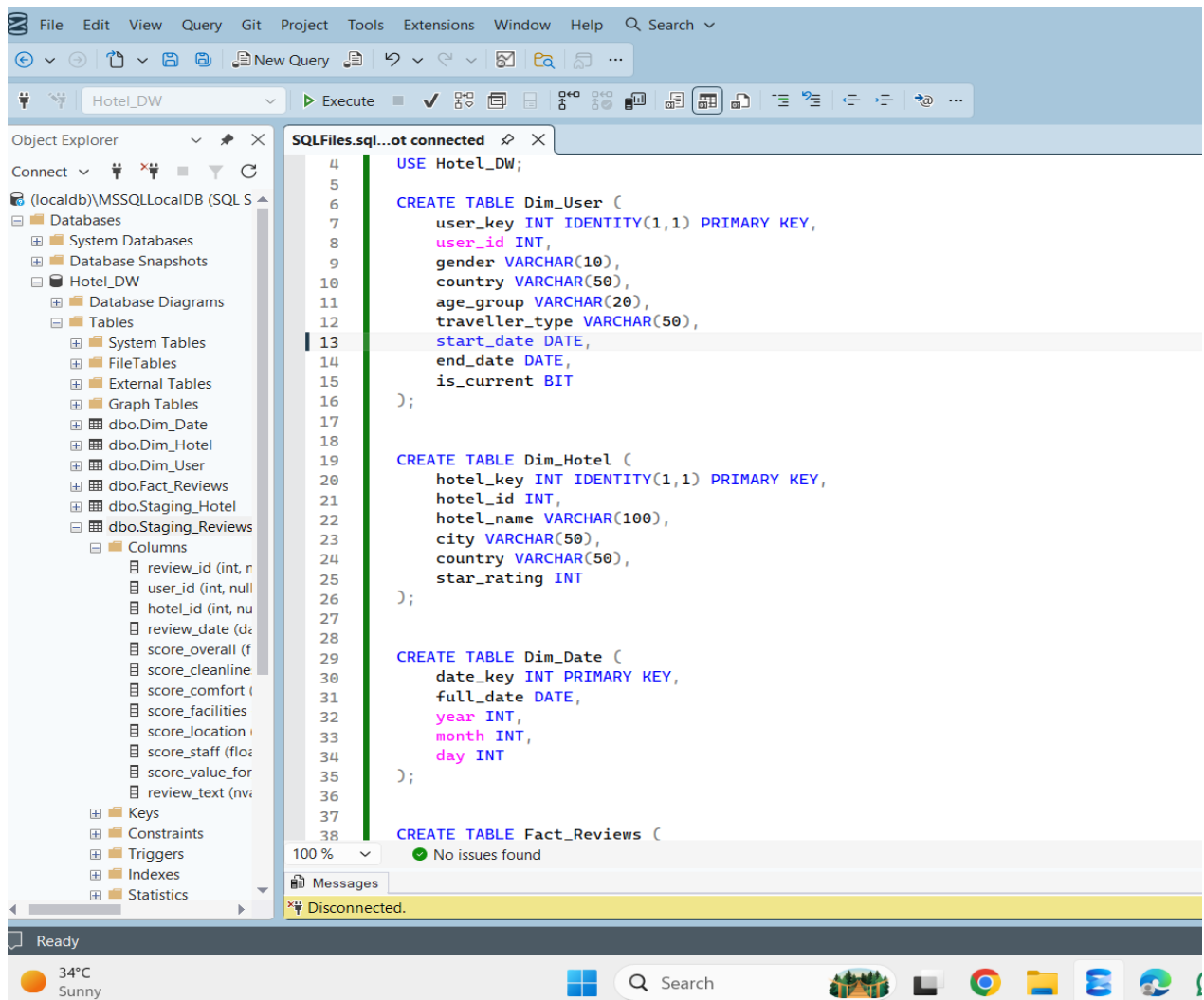


4.4 SQL Implementation

All tables were created using SQL Server scripts. Key implementation highlights:

Creating Staging Tables





5 ETL Development

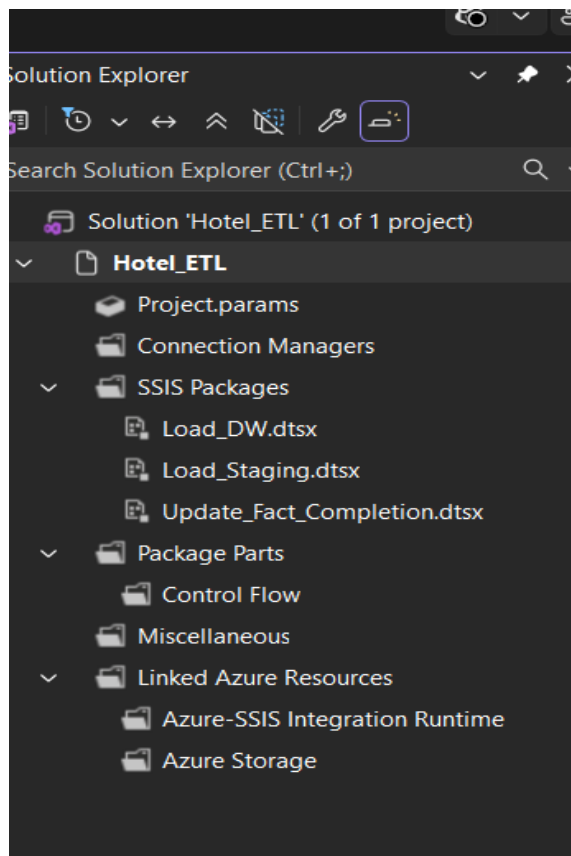
5.1 ETL Overview

The ETL process was implemented using SQL Server Integration Services (SSIS) in Visual Studio. The objective of the ETL process was to extract data from multiple heterogeneous sources (CSV and Excel), transform the data into a suitable format, and load it into the data warehouse following a star schema design. The ETL process was divided into two main stages:

1. Loading data into staging tables
2. Loading dimension and fact tables from staging

Each ETL task was implemented using Data Flow Tasks, incorporating transformations such as Data Conversion, Derived Columns, and Lookups.

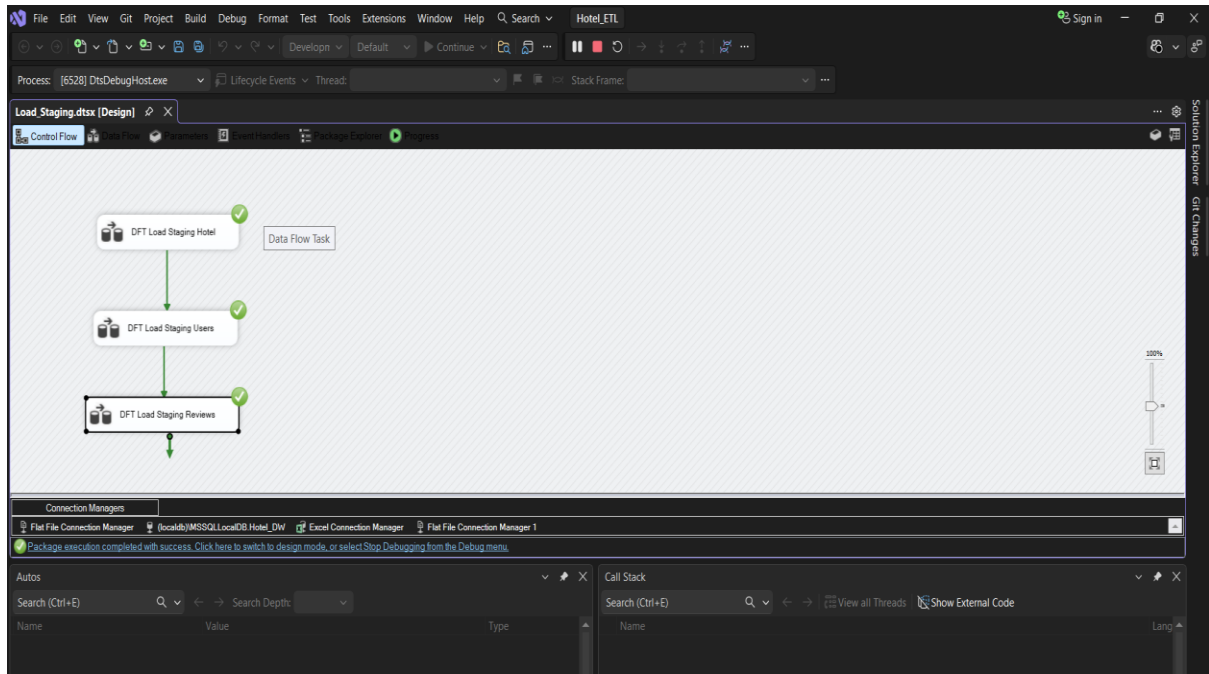
5.2 SSIS Project Structure



The SSIS project structure consists of multiple packages designed to handle different stages of the ETL process. The *Load_Staging* package loads raw data into staging tables, *Load_DW* populates dimension and fact tables, and *Update_Fact_Completion* updates the accumulating fact columns. This modular design improves maintainability and allows independent execution of each ETL phase.

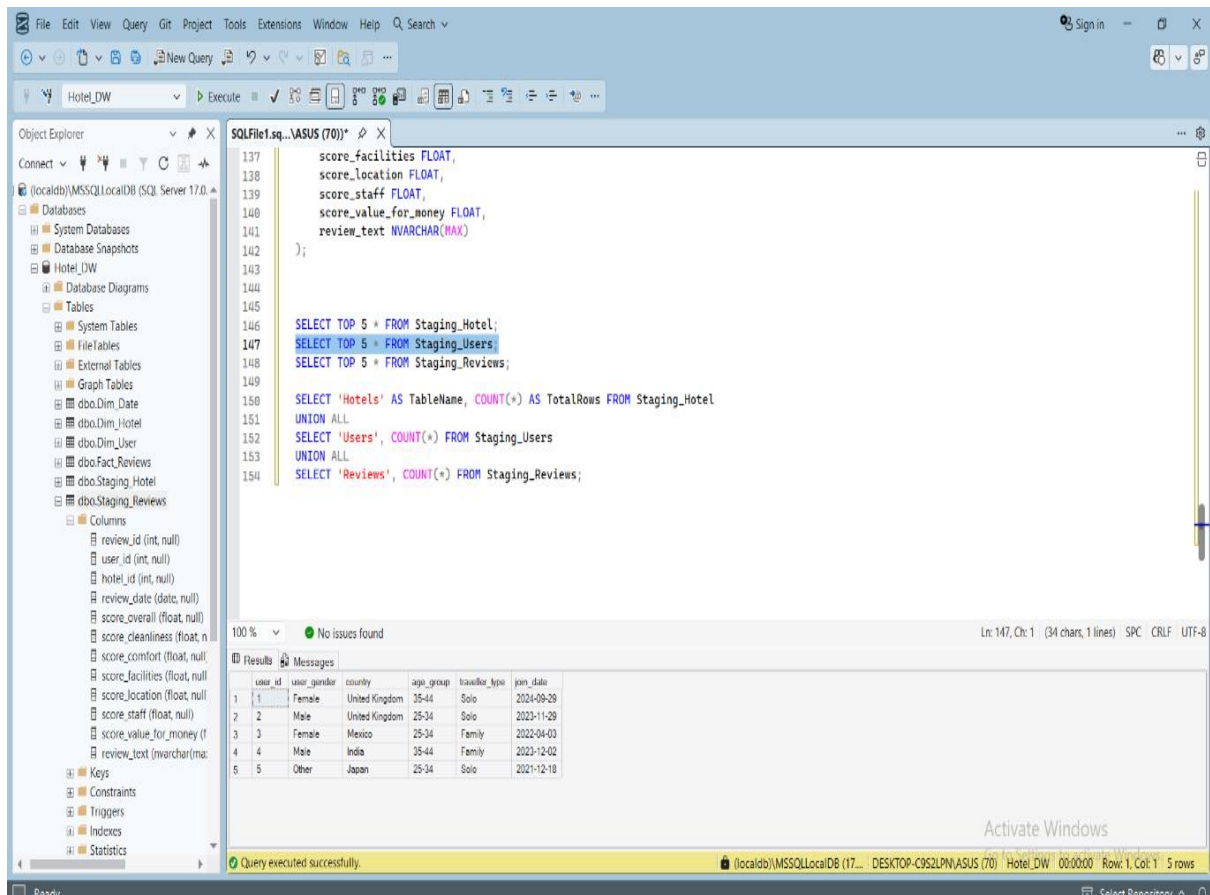
5.3 Load Staging Tables

The staging tables were loaded by extracting raw data from multiple source formats including CSV and Excel files. The data was inserted into staging tables without major transformations to preserve the original structure. This step acts as an intermediate layer to isolate source data from the data warehouse. It also allows easier debugging and reloading during ETL execution.



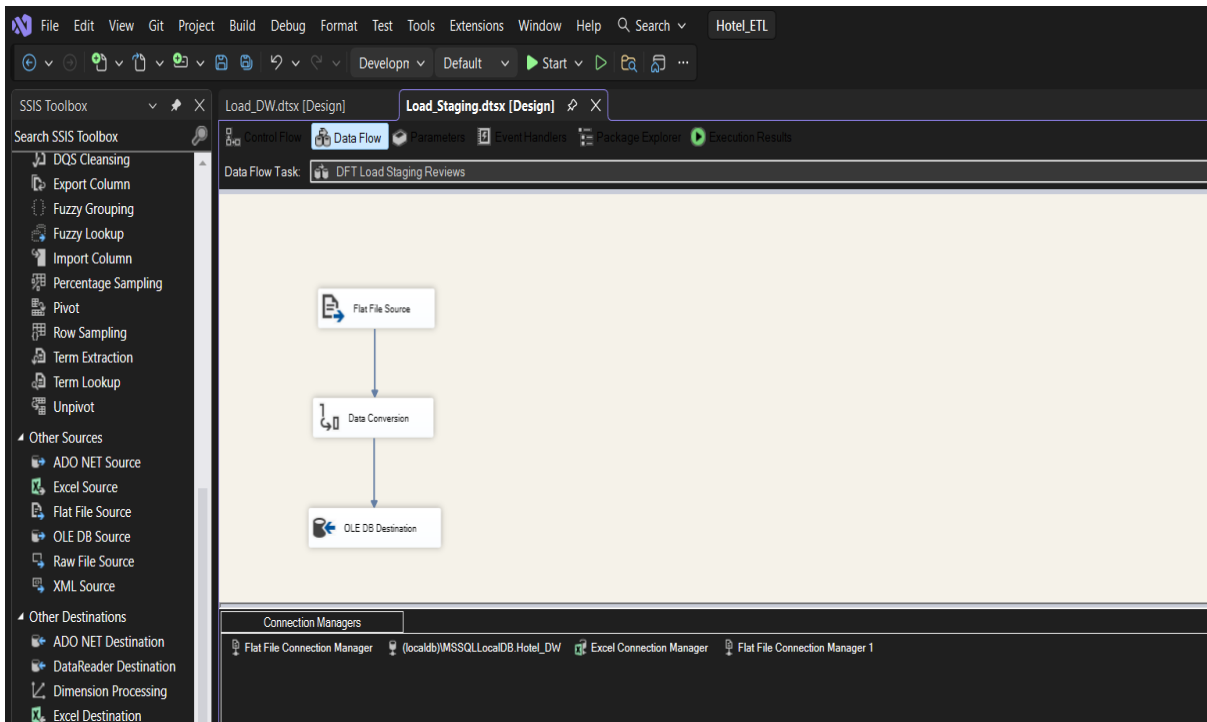
```
SQLFile1.sql... \ASUS (70)) *  
137 score_facilities FLOAT,  
138 score_location FLOAT,  
139 score_staff FLOAT,  
140 score_value_for_money FLOAT,  
141 review_text NVARCHAR(MAX)  
142 );  
143  
144  
145  
146 SELECT TOP 5 * FROM Staging_Hotel;  
147 SELECT TOP 5 * FROM Staging_Users;  
148 SELECT TOP 5 * FROM Staging_Reviews;  
149  
150  
151 SELECT 'Hotels' AS TableName, COUNT(*) AS TotalRows FROM Staging_Hotel  
152 UNION ALL  
153 SELECT 'Users', COUNT(*) FROM Staging_Users  
154 UNION ALL  
155 SELECT 'Reviews', COUNT(*) FROM Staging_Reviews;  
156
```

hotel_id	hotel_name	city	country	star_rating	lat	lon	cleanliness_base	comfort_base	facilities_base	location_base	staff_base	value_for_money_base
1	The Azure Tower	New York	United States	5	40.758	-73.9855	9.1	8.8	8.9	9.5	8.6	8
2	The Royal Compass	London	United Kingdom	5	51.5072	0.1276	9	9.2	8.8	9.4	9	7.9
3	L'Hotel Palace	Paris	France	5	48.8566	2.3522	9.8	9.4	8.7	9.6	9.3	8.1
4	Kyoto Grand	Tokyo	Japan	5	35.6895	139.6817	9.8	9	9.3	9.5	9.5	9.2
5	The Golden Oasis	Dubai	United Arab Emirates	5	25.2769	55.2962	9.3	9.5	9.6	8.9	9.4	8.5



Loading Staging_Reviews Table

The Staging_Reviews table was loaded using a Flat File Source to extract data from the CSV file. A Data Conversion transformation was applied to convert data types such as text and dates into compatible SQL Server formats. The transformed data was then loaded into the staging table using an OLE DB Destination. This process ensures that raw review data is properly formatted before further transformations.



Object Explorer: (localdb)\MSSQLLocalDB (SQL Server 17.0) > Databases > Hotel_DW > Tables > dbo.Staging_Reviews

Columns: review_id (int, null), user_id (int, null), hotel_id (int, null), review_date (date, null), score_overall (float, null), score_cleanliness (float, null), score_comfort (float, null), score_facilities (float, null), score_location (float, null), score_staff (float, null), score_value_for_money (float, null), review_text (nvarchar(max), null)

SQLFile1.sql...JASUS (70) *

```

137 score_facilities FLOAT,
138 score_location FLOAT,
139 score_staff FLOAT,
140 score_value_for_money FLOAT,
141 review_text NVARCHAR(MAX)
142 };
143
144
145
146 SELECT TOP 5 * FROM Staging_Hotel;
147 SELECT TOP 5 * FROM Staging_Users;
148 SELECT TOP 5 * FROM Staging_Reviews;
149
150 SELECT 'Hotels' AS TableName, COUNT(*) AS TotalRows FROM Staging_Hotel
151 UNION ALL
152 SELECT 'Users', COUNT(*) FROM Staging_Users
153 UNION ALL
154 SELECT 'Reviews', COUNT(*) FROM Staging_Reviews;

```

100% No issues found Ltr 148, Cht 1 (36 chars, 1 lines) SPC CRLF UTF-8

	review_id	user_id	hotel_id	review_date	score_overall	score_cleanliness	score_comfort	score_facilities	score_location	score_staff	score_value_for_money	review_text
1	644	1346	7	2023-09-10	8.8	8.8	9.2	8.2	9.6	9	8	Almost lost area. Most room land language thank L...
2	645	661	6	2024-12-15	9	9.3	8.9	8.7	9.3	9.2	8.5	Board later ball imagine rest. System even develop...
3	646	1391	8	2024-04-16	8.8	8.7	8.7	8.3	9.2	9.4	8.9	Suddenly west election fine offer happy. Possible m...
4	647	1453	19	2022-07-15	8.9	8.9	8.7	9.4	9.3	8.8	7.8	But detail approach fine. Whom natural prevent spo...
5	648	1334	17	2024-07-20	8.9	9.2	9.8	8.1	8.7	9.1	8.2	Soon her impact rock like space. Class general test.

Query executed successfully. (localdb)\MSSQLLocalDB (17.0) DESKTOP-C9S2LPN\JASUS (70) Hotel_DW 00:00:00 Row: 1, Col: 1 5 rows

Data Validation

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the 'Object Explorer' with the 'Hotel_DW' database selected. The right pane shows a SQL query window with the following code:

```
137 score_facilities FLOAT,  
138 score_location FLOAT,  
139 score_staff FLOAT,  
140 score_value_for_money FLOAT,  
141 review_text NVARCHAR(MAX)  
142 );  
143  
144  
145  
146 SELECT TOP 5 * FROM Staging_Hotel;  
147 SELECT TOP 5 * FROM Staging_Users;  
148 SELECT TOP 5 * FROM Staging_Reviews;  
149  
150 SELECT 'Hotels' AS TableName, COUNT(*) AS TotalRows FROM Staging_Hotel  
151 UNION ALL  
152 SELECT 'Users', COUNT(*) FROM Staging_Users  
153 UNION ALL  
154 SELECT 'Reviews', COUNT(*) FROM Staging_Reviews;
```

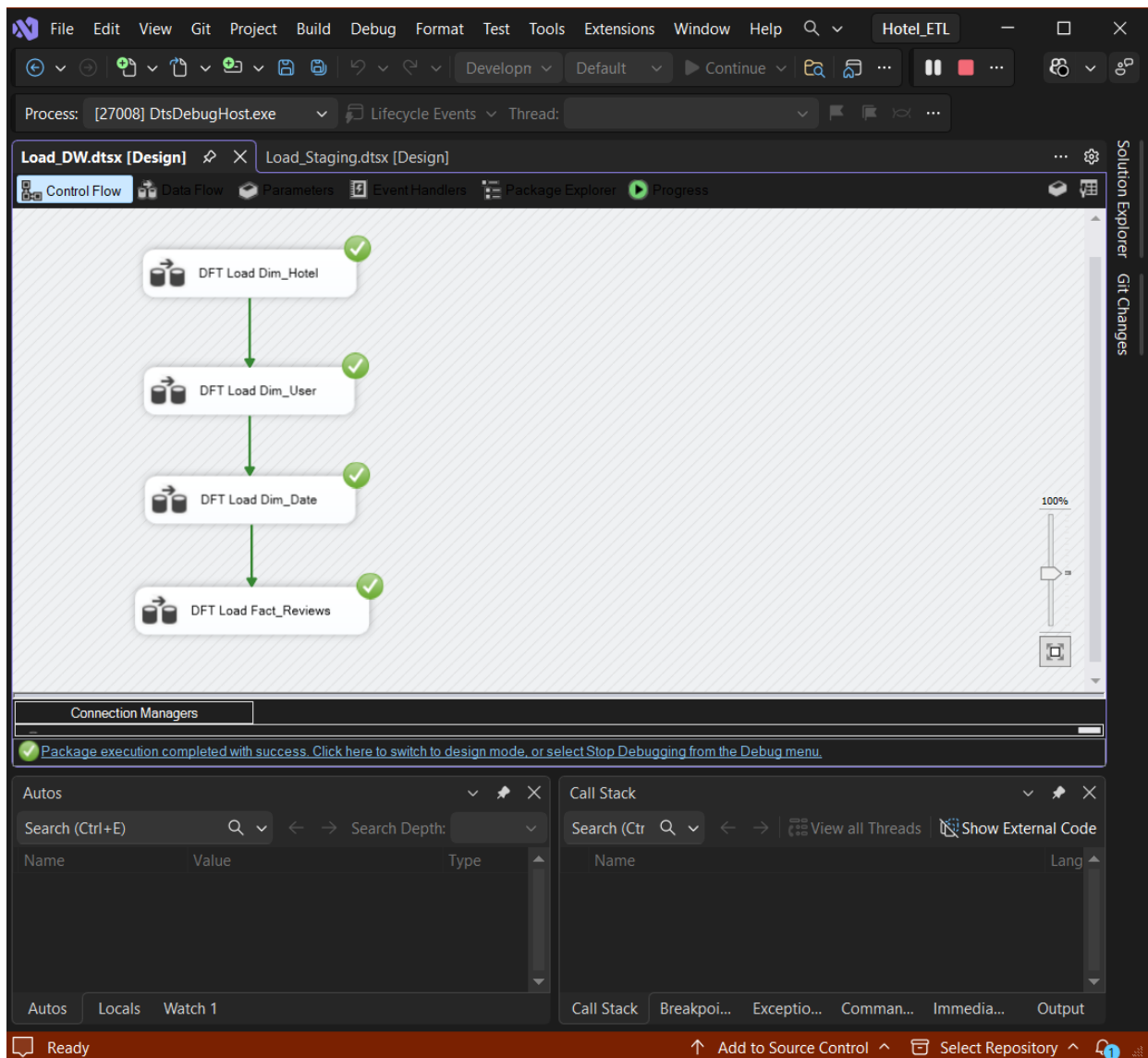
The bottom pane shows the 'Results' tab with the following data:

TableName	TotalRows
Hotels	150
Users	11954
Reviews	50000

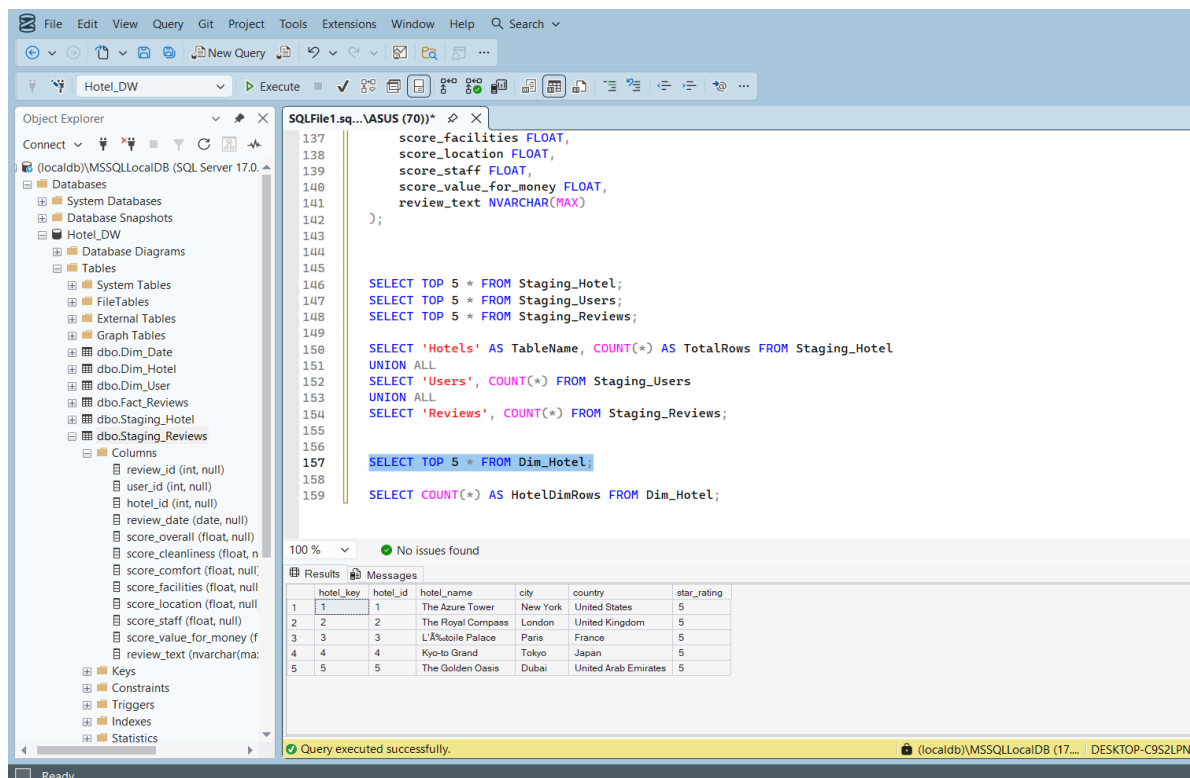
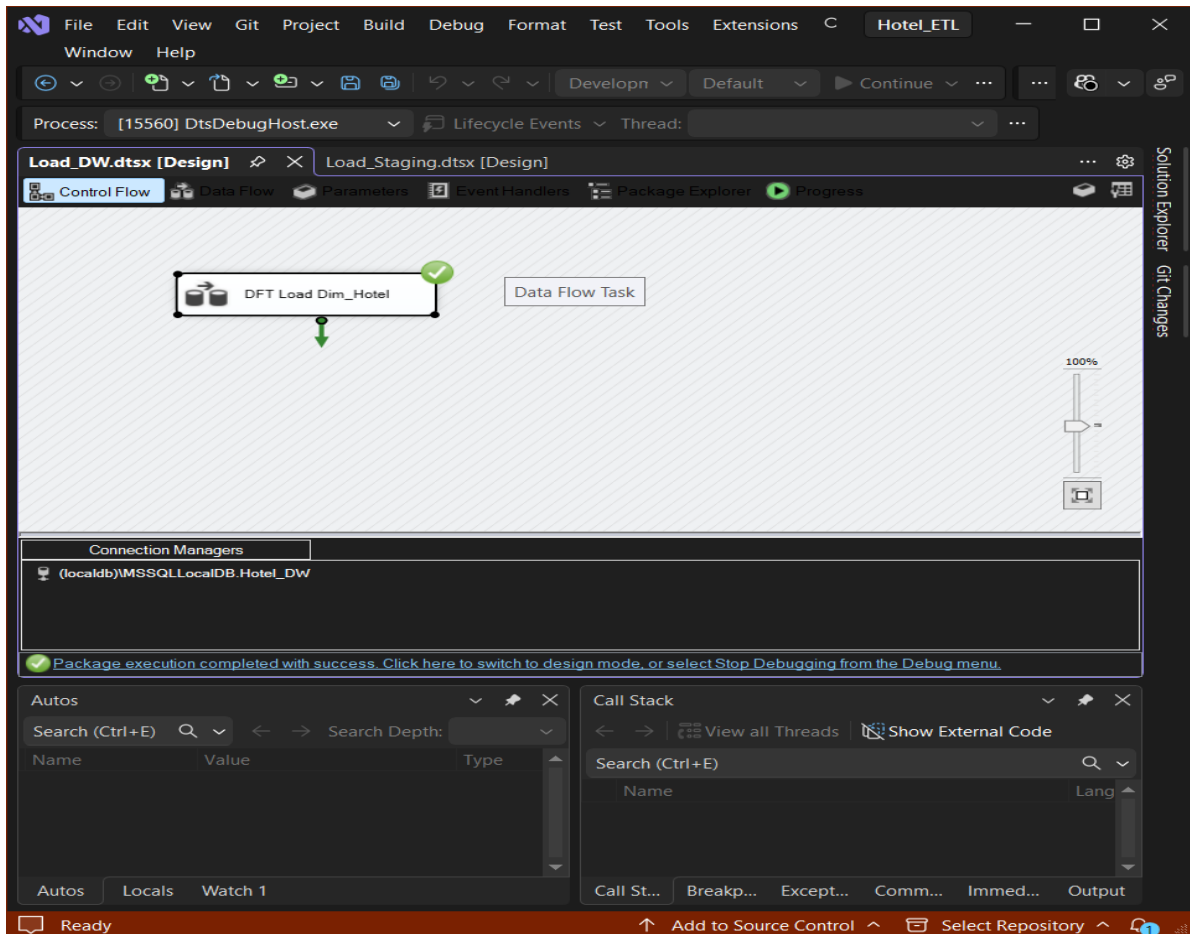
The status bar at the bottom indicates 'Query executed successfully.' and 'No issues found'.

5.4 Load Dimensions Tables

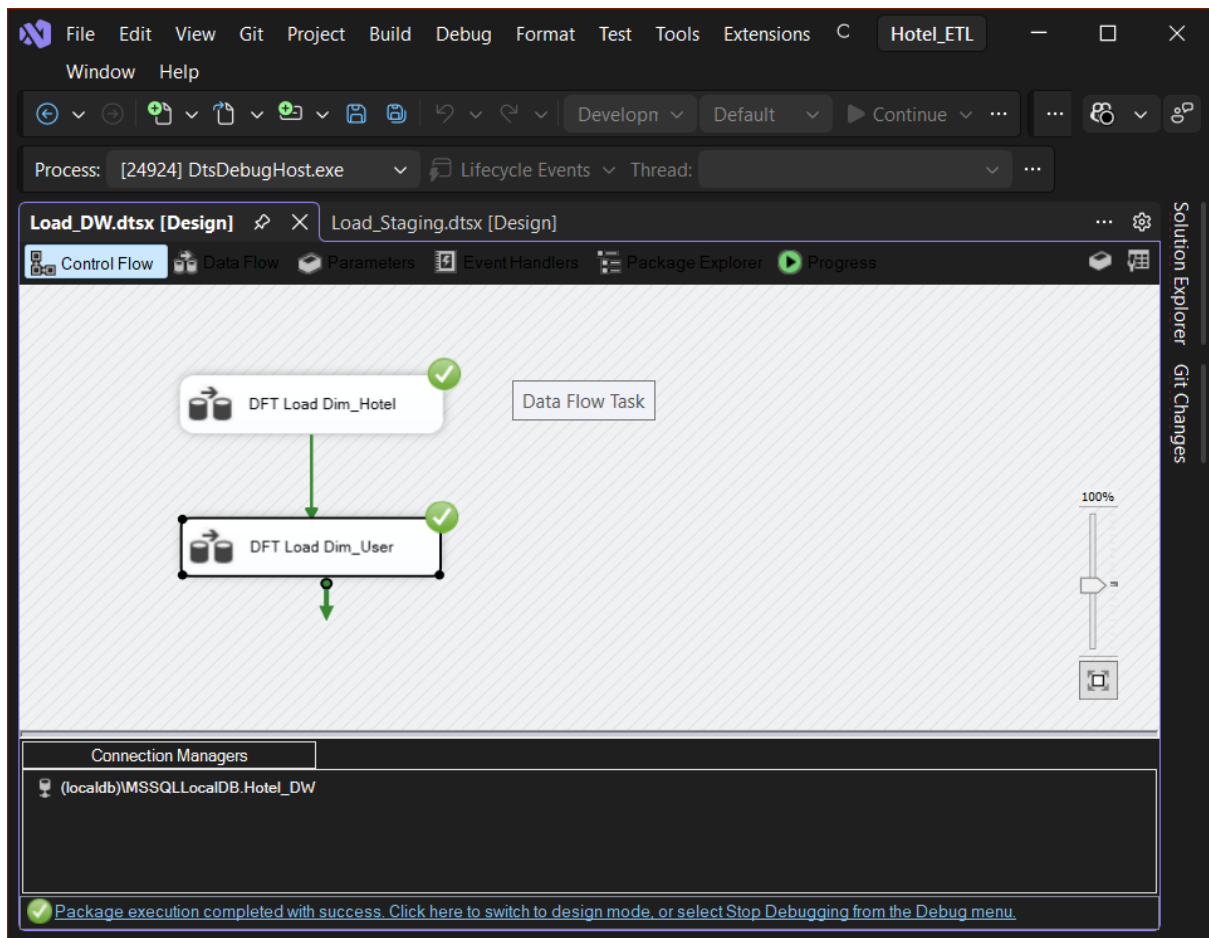
Dimension tables were populated from staging tables after applying necessary transformations such as data cleaning, type conversion, and duplicate removal. Each dimension was designed with surrogate keys to support efficient joins. The Dim_Date table was derived from review dates, while Dim_User and Dim_Hotel were directly loaded from staging data. This step ensures structured and descriptive data for analysis.



Dim Hotel



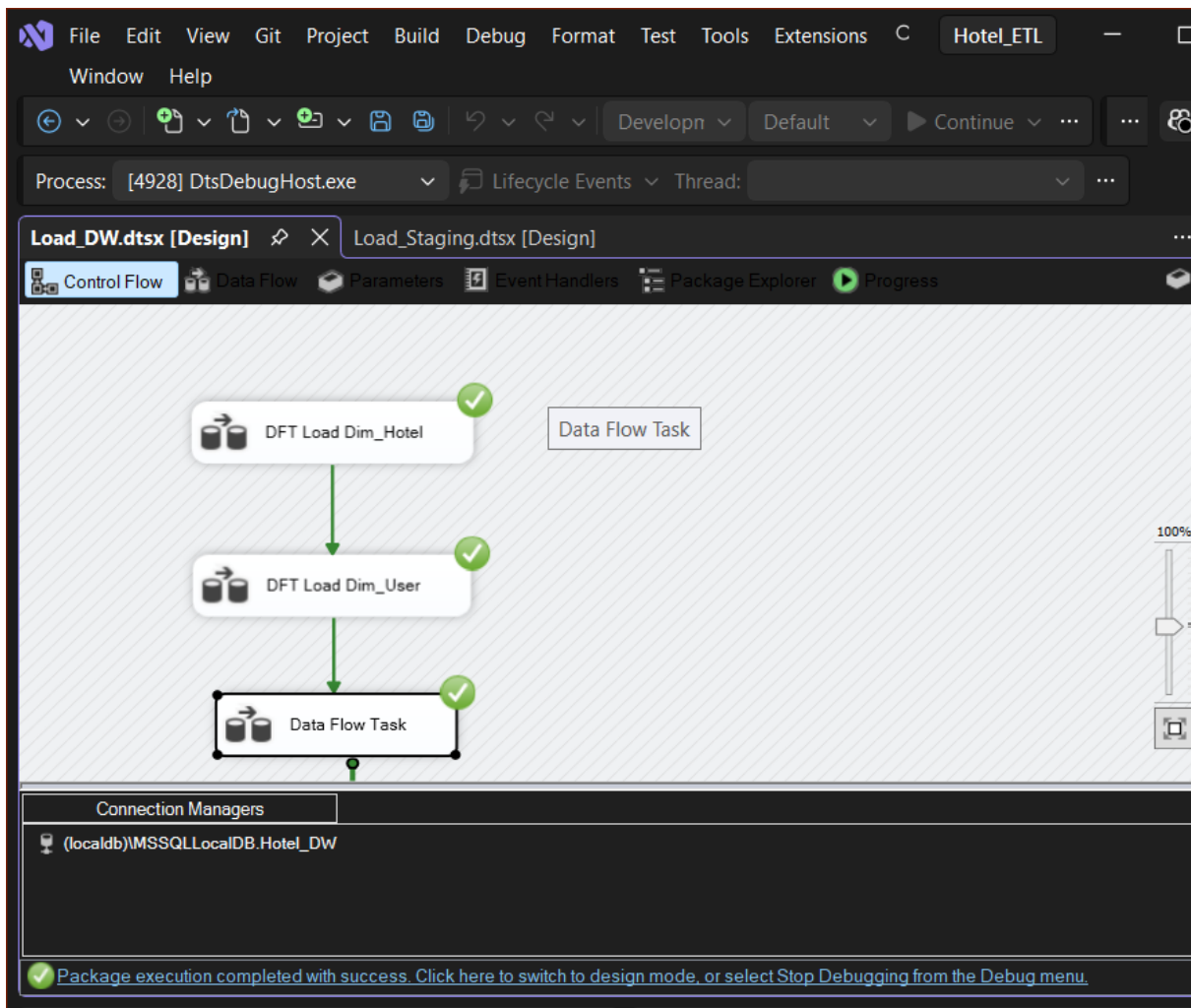
Dim User



The screenshot shows the SQL Server Enterprise Manager (SEM) interface. The 'Object Explorer' on the left shows the 'Hotel_DW' database. The 'Query Explorer' on the right shows a query executed against the 'Hotel_DW' database. The query results are displayed in a table with 5 rows and 10 columns.

user_key	user_id	gender	country	age_group	traveller_type	start_date	end_date	is_current
1	1	Female	United Kingdom	35-44	Solo	2026-03-25	NULL	1
2	2	Male	United Kingdom	25-34	Solo	2026-03-25	NULL	1
3	3	Female	Mexico	25-34	Family	2026-03-25	NULL	1
4	4	Male	India	35-44	Family	2026-03-25	NULL	1
5	5	Other	Japan	25-34	Solo	2026-03-25	NULL	1

Dim Date



The screenshot shows the SQL Server Enterprise Manager interface. The **Hotel_DW** database is selected in the **Object Explorer**. The **dbo** schema is expanded, showing the **Dim_Date** table. The **Columns** tab for **Dim_Date** is selected, showing the following columns:

- date_key** (int, null)
- user_id** (int, null)
- hotel_id** (int, null)
- review_date** (date, null)
- score_overall** (float, null)
- score_cleanliness** (float, null)
- score_comfort** (float, null)
- score_facilities** (float, null)
- score_location** (float, null)
- score_staff** (float, null)
- score_value_for_money** (float, null)
- review_text** (nvarchar(max), null)

The **Results** tab shows the query results for the **Dim_Date** table. The query is:

```
SELECT TOP 5 * FROM Dim_Date ORDER BY full_date;
```

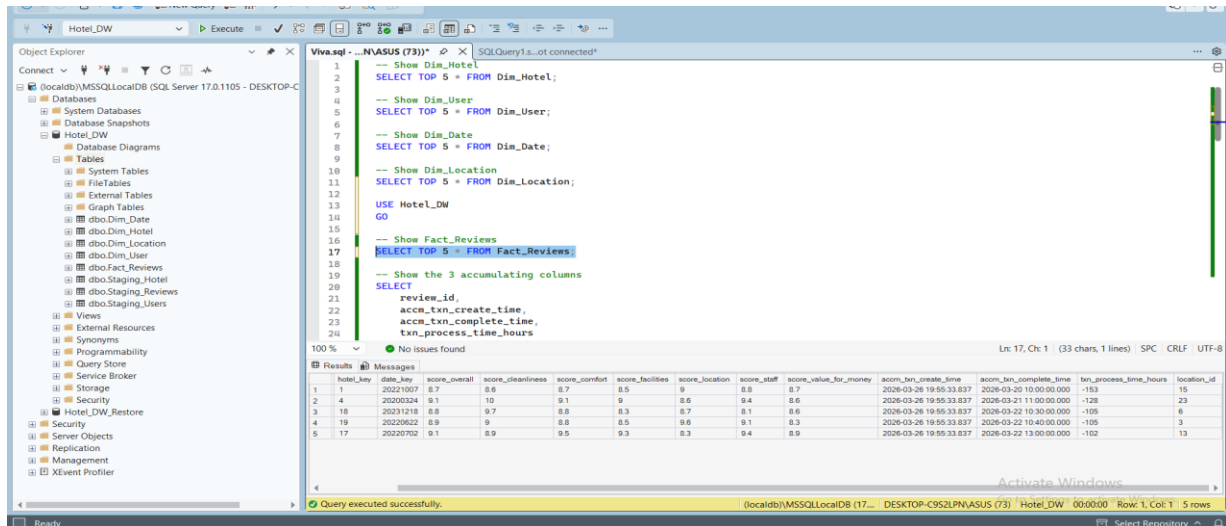
The results are as follows:

date_key	full_date	year	month	day
19000101	1900-01-01	1900	1	1
20200101	2020-01-01	2020	1	1
20200102	2020-01-02	2020	1	2
20200103	2020-01-03	2020	1	3
20200104	2020-01-04	2020	1	4

The status bar indicates: **Query executed successfully.**

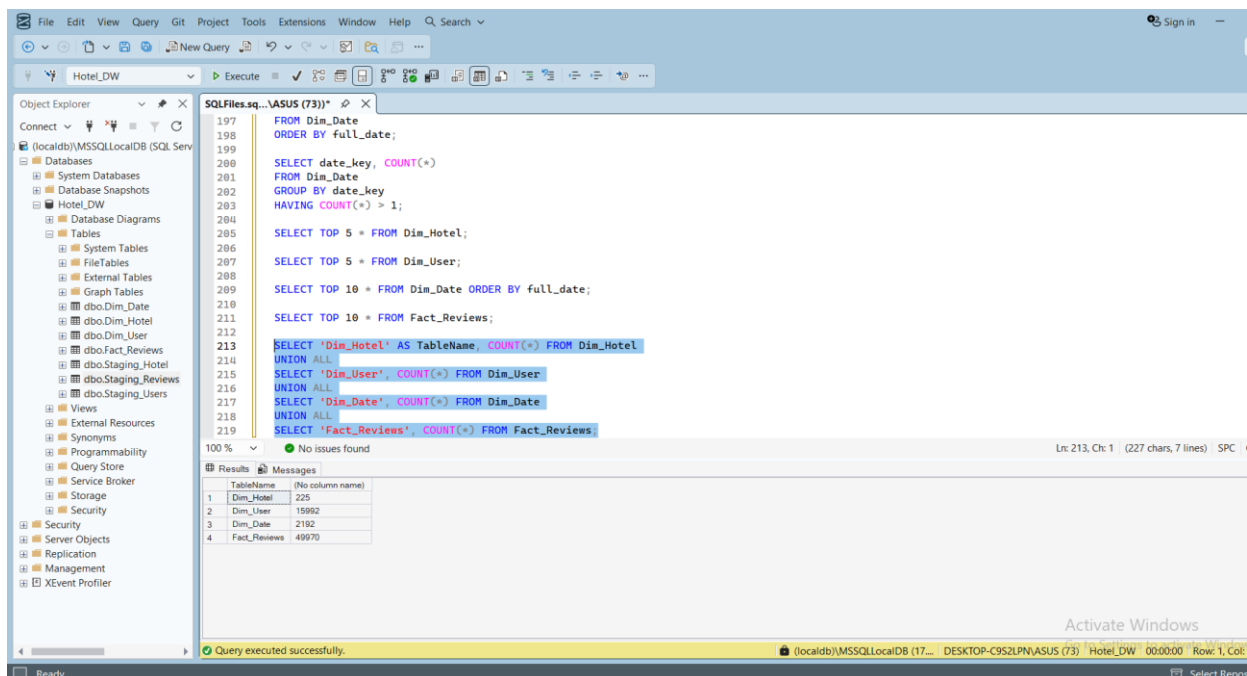
5.5 Load Fact Table

The fact table was loaded using data from the staging reviews table after applying transformations and lookups. Foreign keys were mapped to dimension tables using lookup transformations to ensure referential integrity. Additional derived columns such as date_key were created during the process. This step integrates all dimensions with measurable data to support analytical queries.



Data Validation

After ETL execution, validation queries were executed to ensure correctness.

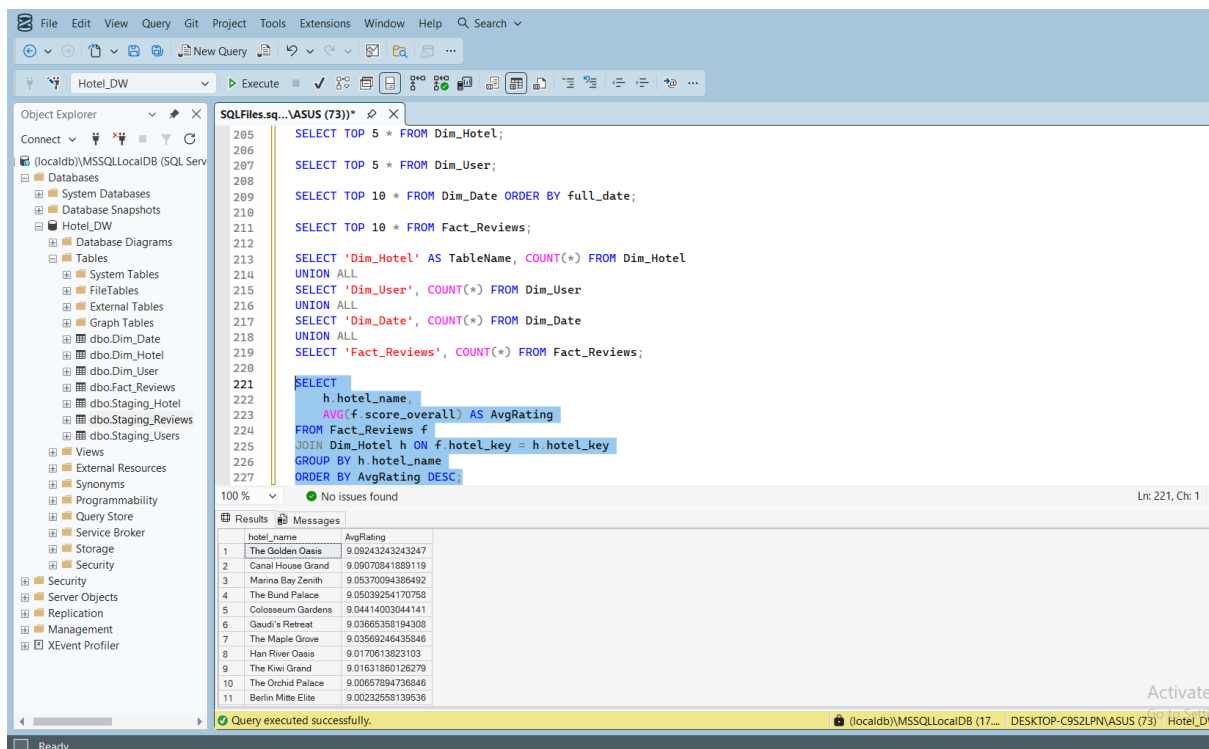


6 Accumulating Fact Table

The Fact_Reviews table was extended to support an accumulating snapshot model. This allows tracking of the lifecycle of a transaction by storing timestamps such as creation time and completion time. Additionally, the processing duration was calculated to support time-based analysis.

6.1 Fact Table Analysis

An analytical query was performed on the Fact_Reviews table by joining it with the Dim_Hotel table. The average overall rating was calculated for each hotel and sorted in descending order. This demonstrates how the data warehouse supports business intelligence queries and enables meaningful insights from the stored data.



The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the 'Object Explorer' with the 'Hotel_DW' database selected. The right pane shows a SQL query window with the following code:

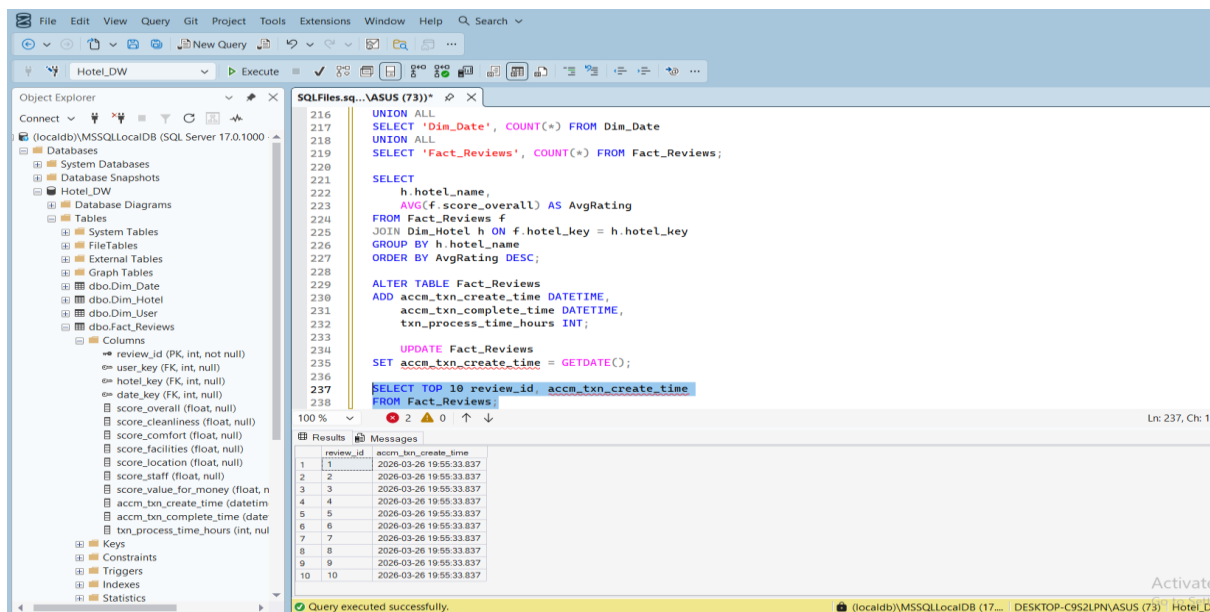
```
205 SELECT TOP 5 * FROM Dim_Hotel;
206
207 SELECT TOP 5 * FROM Dim_User;
208
209 SELECT TOP 10 * FROM Dim_Date ORDER BY full_date;
210
211 SELECT TOP 10 * FROM Fact_Reviews;
212
213 SELECT 'Dim_Hotel' AS TableName, COUNT(*) FROM Dim_Hotel
214 UNION ALL
215 SELECT 'Dim_User', COUNT(*) FROM Dim_User
216 UNION ALL
217 SELECT 'Dim_Date', COUNT(*) FROM Dim_Date
218 UNION ALL
219 SELECT 'Fact_Reviews', COUNT(*) FROM Fact_Reviews;
220
221 SELECT
222     h.hotel_name,
223     AVG(f.score_overall) AS AvgRating
224 FROM Fact_Reviews f
225 JOIN Dim_Hotel h ON f.hotel_key = h.hotel_key
226 GROUP BY h.hotel_name
227 ORDER BY AvgRating DESC;
```

The 'Results' pane shows the output of the query, displaying a table with two columns: 'hotel_name' and 'AvgRating'. The results are sorted in descending order of the average rating.

hotel_name	AvgRating
1 The Golden Oasis	9.09243243243247
2 Canal House Grand	9.09070841889119
3 Marina Bay Zenith	9.05370094386492
4 The Bund Palace	9.05039254170758
5 Colosseum Gardens	9.04414003044141
6 Gaudi's Retreat	9.03665358194308
7 The Maple Grove	9.03569246435846
8 Han River Oasis	9.0170613823103
9 The Kiwi Grand	9.01631860126279
10 The Orchid Palace	9.00657894736846
11 Berlin Mitte Elite	9.00232558139536

6.2 Accumulating Columns & Update

Additional columns were added to the Fact_Reviews table to support the accumulating fact model, including transaction creation time, completion time, and processing duration. The creation time was populated using the GETDATE() function. A SELECT query was used to verify that the new columns were correctly updated. This enables tracking of the transaction lifecycle within the fact table.



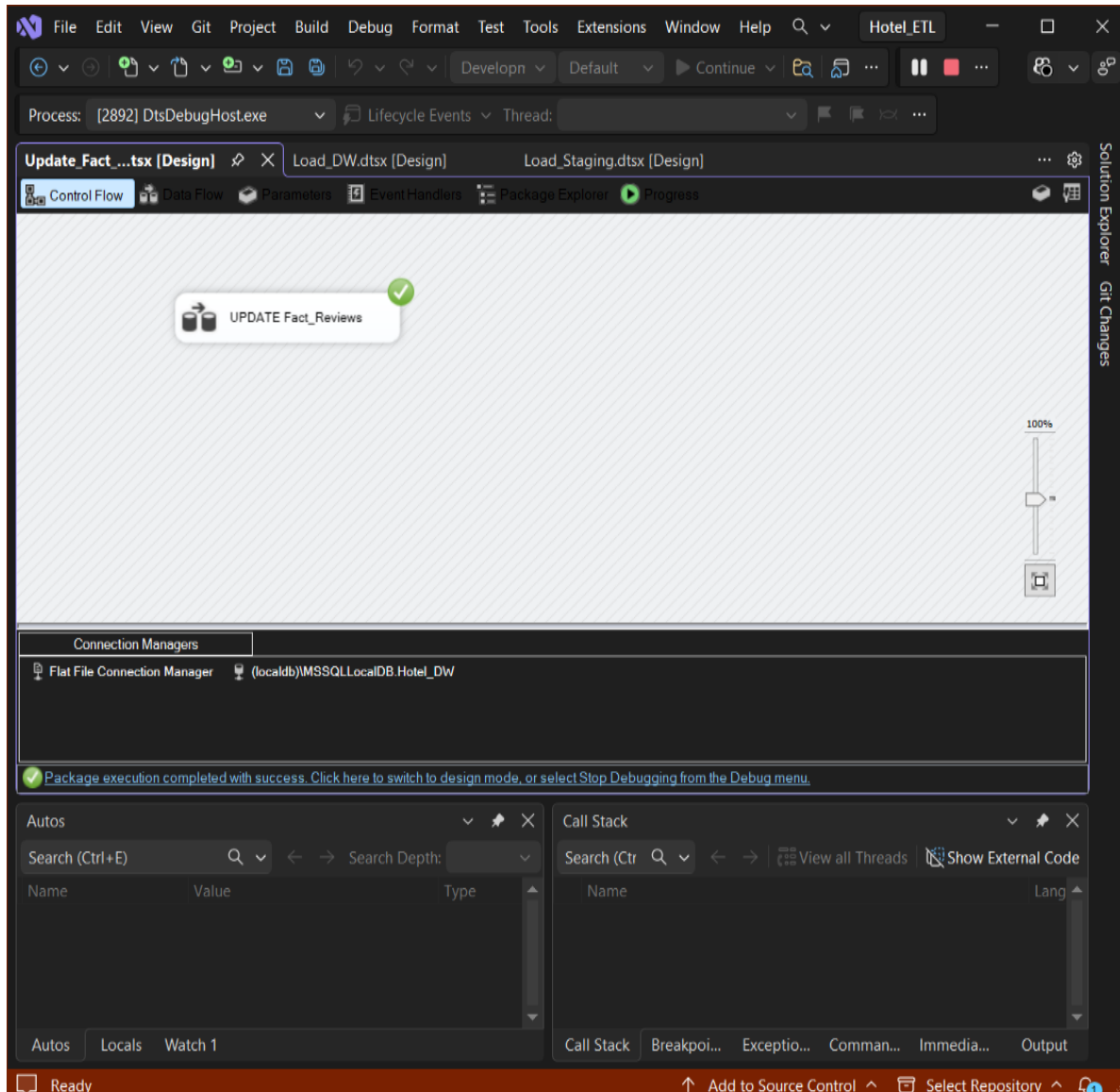
6.3 Creation of Completion Dataset

A completion dataset was created in Excel containing review IDs and corresponding transaction completion timestamps. This dataset simulates the final stage of a transaction lifecycle. It is later used as an external source to update the fact table. This approach reflects real-world scenarios where completion data arrives separately.

review_id	accm_txn_complete_time
1	2026-03-20 10:00:00
2	2026-03-21 11:00:00
3	2026-03-22 10:30:00
4	2026-03-22 10:40:00
5	2026-03-22 13:00:00
6	2026-03-23 14:00:01
7	2026-03-24 09:45:02
8	2026-03-24 10:50:03
9	2026-03-24 14:00:04
10	2026-03-25 11:00:05
11	2026-03-25 13:00:15
12	2026-03-26 08:00:50
13	2026-03-26 11:22:08
14	2026-03-26 15:00:09
15	2026-03-27 07:32:10
16	2026-03-27 13:00:11
17	2026-03-28 10:00:41
18	2026-03-28 16:42:13
19	2026-03-29 06:52:14
20	2026-03-29 09:00:15

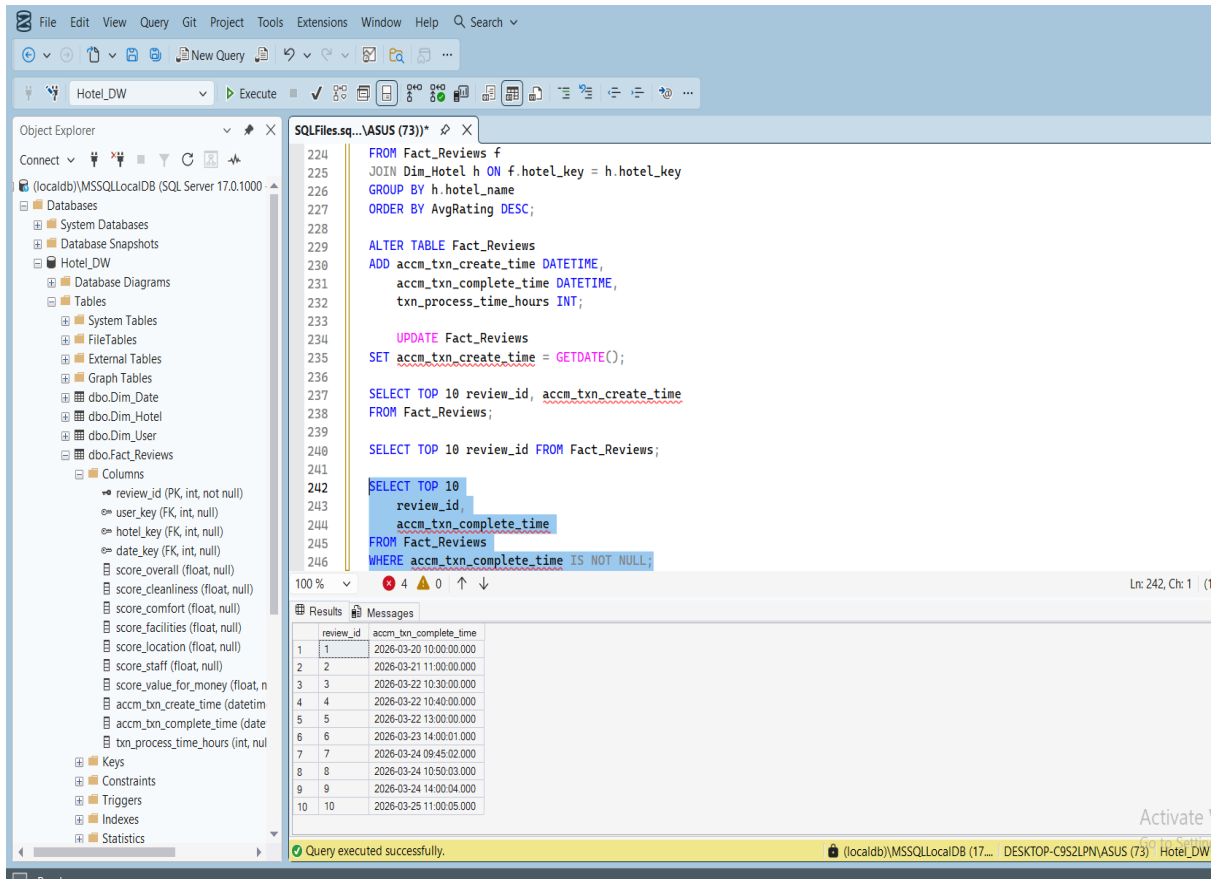
6.4 SSIS Package for Updating Fact Table

A separate SSIS package was developed to update the Fact_Reviews table using the completion dataset. The package reads data from the flat file and updates the corresponding records in the data warehouse. The successful execution (green tick) confirms that the update process was completed without errors. This step enables incremental updates to the fact table.



6.5 Updating Completion Time in Fact Table

The `accm_txn_complete_time` column was populated using the completion dataset through the SSIS update process. A `SELECT` query was executed to verify that the completion times were correctly updated in the `Fact_Reviews` table. This ensures that each transaction now contains both start and completion timestamps



The screenshot displays the SQL Server Enterprise Manager interface. The Object Explorer on the left shows the `Hotel_DW` database structure, including tables like `Fact_Reviews`. The central pane shows a SQL query being executed. The query includes a `SELECT` statement to verify the completion times in the `Fact_Reviews` table.

```
224 FROM Fact_Reviews f
225 JOIN Dim_Hotel h ON f.hotel_key = h.hotel_key
226 GROUP BY h.hotel_name
227 ORDER BY AvgRating DESC;
228
229 ALTER TABLE Fact_Reviews
230 ADD accm_txn_create_time DATETIME,
231     accm_txn_complete_time DATETIME,
232     txn_process_time_hours INT;
233
234 UPDATE Fact_Reviews
235 SET accm_txn_create_time = GETDATE();
236
237 SELECT TOP 10 review_id, accm_txn_create_time
238 FROM Fact_Reviews;
239
240 SELECT TOP 10 review_id FROM Fact_Reviews;
241
242 SELECT TOP 10
243     review_id,
244     accm_txn_complete_time
245 FROM Fact_Reviews
246 WHERE accm_txn_complete_time IS NOT NULL;
```

The Results pane at the bottom shows the output of the `SELECT` query, displaying 10 rows of data with `review_id` and `accm_txn_complete_time`.

review_id	accm_txn_complete_time
1	2026-03-20 10:00:00.000
2	2026-03-21 11:00:00.000
3	2026-03-22 10:30:00.000
4	2026-03-22 10:40:00.000
5	2026-03-22 13:00:00.000
6	2026-03-23 14:00:01.000
7	2026-03-24 09:45:02.000
8	2026-03-24 10:50:03.000
9	2026-03-24 14:00:04.000
10	2026-03-25 11:00:05.000

6.6 Calculating Processing Time

The transaction processing time was calculated using the `DATEDIFF` function, which computes the difference between creation time and completion time in hours. The calculated values were stored in the `txn_process_time_hours` column. This enables performance analysis of transaction durations within the data warehouse.

Object Explorer: (localdb)\MSSQLLocalDB (SQL Server 17.0.1000)

- Databases
 - System Databases
 - Database Snapshots
 - Hotel_DW
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.Dim_Date
 - dbo.Dim_Hotel
 - dbo.Dim_User
 - dbo.Fact_Reviews
 - Columns
 - review_id (PK, int, not null)
 - user_key (FK, int, null)
 - hotel_key (FK, int, null)
 - date_key (FK, int, null)
 - score_overall (float, null)
 - score_cleanliness (float, null)
 - score_comfort (float, null)
 - score_location (float, null)
 - score_staff (float, null)
 - score_value_for_money (float, null)
 - accm_txn_create_time (datetime)
 - accm_txn_complete_time (datetime)
 - txn_process_time_hours (int, null)
 - Keys
 - Constraints
 - Triggers
 - Indexes
 - Statistics

SQLFiles.sql... \ASUS (73))

```

241
242
243 SELECT TOP 10
244     review_id,
245     accm_txn_create_time,
246     accm_txn_complete_time
247 FROM Fact_Reviews
248 WHERE accm_txn_complete_time IS NOT NULL;
249
250 UPDATE Fact_Reviews
251 SET txn_process_time_hours =
252     DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time)
253 WHERE accm_txn_complete_time IS NOT NULL;
254
255 SELECT TOP 20
256     review_id,
257     accm_txn_create_time,
258     accm_txn_complete_time,
259     txn_process_time_hours
260 FROM Fact_Reviews
261 WHERE accm_txn_complete_time IS NOT NULL;
  
```

Results

review_id	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	2026-03-26 19:55:33.837	2026-03-20 10:00:00.000	-153
2	2026-03-26 19:55:33.837	2026-03-21 11:00:00.000	-128
3	2026-03-26 19:55:33.837	2026-03-22 10:30:00.000	-105
4	2026-03-26 19:55:33.837	2026-03-22 10:40:00.000	-105
5	2026-03-26 19:55:33.837	2026-03-22 13:00:00.000	-102
6	2026-03-26 19:55:33.837	2026-03-23 14:00:01.000	-77
7	2026-03-26 19:55:33.837	2026-03-24 09:45:02.000	-58
8	2026-03-26 19:55:33.837	2026-03-24 10:50:03.000	-57
9	2026-03-26 19:55:33.837	2026-03-24 14:00:04.000	-53
10	2026-03-26 19:55:33.837	2026-03-25 11:00:05.000	-32
11	2026-03-26 19:55:33.837	2026-03-25 13:00:15.000	-30

Query executed successfully.

6.7 Validation of Accumulating Fact Data

Object Explorer: (localdb)\MSSQLLocalDB (SQL Server 17.0.1000)

- Databases
 - System Databases
 - Database Snapshots
 - Hotel_DW
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.Dim_Date
 - dbo.Dim_Hotel
 - dbo.Dim_User
 - dbo.Fact_Reviews
 - Columns
 - review_id (PK, int, not null)
 - user_key (FK, int, null)
 - hotel_key (FK, int, null)
 - date_key (FK, int, null)
 - score_overall (float, null)
 - score_cleanliness (float, null)
 - score_comfort (float, null)
 - score_location (float, null)
 - score_staff (float, null)
 - score_value_for_money (float, null)
 - accm_txn_create_time (datetime)
 - accm_txn_complete_time (datetime)
 - txn_process_time_hours (int, null)
 - Keys
 - Constraints
 - Triggers
 - Indexes
 - Statistics

SQLFiles.sql... \ASUS (73))

```

245 FROM Fact_Reviews
246 WHERE accm_txn_complete_time IS NOT NULL;
247
248
249 UPDATE Fact_Reviews
250 SET txn_process_time_hours =
251     DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time)
252 WHERE accm_txn_complete_time IS NOT NULL;
253
254
255 SELECT TOP 20
256     review_id,
257     accm_txn_create_time,
258     accm_txn_complete_time,
259     txn_process_time_hours
260 FROM Fact_Reviews
261 WHERE accm_txn_complete_time IS NOT NULL;
262
263
264 SELECT COUNT(*) AS UpdatedRows
265 FROM Fact_Reviews
266 WHERE accm_txn_complete_time IS NOT NULL
267 AND txn_process_time_hours IS NOT NULL;
  
```

Results

UpdatedRows
20

Query executed successfully.

